

当自主开发退化为“点继续”： 一个 AI 终端编排系统的设计、部署与实测反思

网梯 harness 项目组

<https://github.com/auntunt/WhatyTerm>

2026 年 7 月

摘要

大语言模型 (LLM) 驱动的命令编码代理在长程软件开发任务中普遍面临一个被忽视的工程难题：代理会频繁停下来等待人工介入——等待“继续”、等待权限确认、等待菜单选择，或卡死在瞬时错误上。本文介绍 WhatyTerm，一个为消除这些中断点而设计的 AI 终端会话编排系统，并以**真实运行数据**为核心报告其设计意图与实际行为之间的落差。系统设计上包含三项机制：以规则层优先、LLM 兜底的分层决策；联合 Hook 事件、终端抓屏与进程指纹的三通道感知；以及 micro/meso/macro 三层嵌套的自主交付循环。然而，对一次真实部署（跨三天、四个会话、46 次自动决策）的日志分析揭示了一个与设计初衷相悖的现象：**全部 46 次决策 100% 由零 Token 的规则层做出，LLM 兜底层一次也未被触发**；其中 87% 的动作是机械地发送“继续”，仅 1 次注入了真实的任务指令。换言之，一个本应智能编排的自主开发系统，在真实运行中退化成了一个“自动点继续”的宏。我们剖析了这一退化的代码级成因——规则层在检测到空闲提示符时无条件发送“继续”，缺乏“本次继续是否推进了任务”的判断，也没有将该判断移交 LLM 的路径——并据此讨论自主代理 harness 中“规则层与推理层边界”这一更普遍的设计陷阱。本文的立场是：报告一个系统未按预期工作的真实证据与根因，比宣称其设计优越更有价值。

关键词： AI 编码代理；自主软件开发；终端自动化；负面结果；人在环

1 引言

基于大语言模型的命令行编码代理（Claude Code、Codex CLI、Gemini CLI 等）已能在终端中读写代码、执行命令、运行测试并以对话推进任务。但当任务从“回答一个问题”扩展为“完成一个多步骤、跨文件、需反复编译测试的开发目标”时，它们暴露出一个共性瓶颈：无法在无人值守下持续推进。一次长程任务会被大

量“中断点”打断——完成一步后停下等“继续”、执行敏感操作时弹确认框、面临多个方案时列菜单、遇到限流或余额不足时停在错误上。这些中断点使“让代理跑一整晚把功能做完”在实践中难以成立。

WhatyTerm 正是为消除这些中断点而设计的编排系统：它在编码代理与人类之间插入一个自动化监督层，实时感知终端状态、判断代理是否需要外部动作，并代替人类注入相应操作。系统的设计蓝图包含三项机制：规则优先、LLM 兜底的分层决策 (§4.1)；Hook 事件、终端抓屏、进程指纹三通道感知 (§3.2)；以及 micro/meso/macro 三层嵌套自主循环 (§4.3)。这些机制在设计层面自治，也构成了本文第 3-5 节所述的系统。

然而，本文的核心不是宣称这套设计有效，而是报告它在真实运行中并未按预期工作。我们对一次真实部署的完整日志（跨越三天、覆盖四个会话、共 46 次自动决策）做了分析，得到一个与设计初衷直接冲突的结果 (§6)：

- 全部 46 次自动决策 100% 由零 Token 的规则层做出，本应处理复杂场景的 LLM 兜底层**一次也未被触发**；
- 其中 87% (40/46) 的动作只是机械地发送“继续”，仅有 1 次注入了真实的任务指令；
- 与之呼应，系统的自主执行引擎在同期反复产出空输出与执行错误，从未真正交付内容。

也就是说，一个本应“智能编排自主开发”的系统，在真实运行中退化成了一个“自动点继续”的宏。这一现象并非偶然的实现 bug，而是暴露了自主代理 harness 中一个更普遍、更隐蔽的设计陷阱：当廉价的规则层被赋予对高频状态的最终决定权，而缺乏“该动作是否真正推进了任务”的反馈判断时，系统会稳定地收敛到“看起来在工作、实则空转”的局部最优。规则层的高命中率——一个本被当作“成本优势”的指标——恰恰成了智能缺位的伪装。

基于此，本文的贡献重新定位如下：

- 一个**诚实的负面结果 (§6)**。基于真实部署日志，量

化并展示一个自主开发 harness 如何退化为“点继续”宏，包括决策类型分布、规则/LLM 占比与动作构成。

- **退化的代码级根因分析 (§7)**。定位到规则层“检测到空闲提示符即无条件发送继续”的判定路径，说明为何 LLM 兜底层永不触发，并指出这是一个可推广的“规则层—推理层边界”反模式。
- **系统设计的完整披露 (§3–5)**。如实呈现原始设计蓝图（分层决策、三通道感知、三层自主循环、领域策略框架），使读者能对照“设计意图”与“实际行为”的落差。
- **对自主 harness 设计的反思与改进方向 (§9)**，包括如何为规则层引入“任务进度”反馈、如何设定规则层向推理层的强制移交条件。

本文所有定量结论均来自可查证的真实运行日志 (SQLite 历史记录与引擎输出文件)，文中不使用任何未经测量的臆造数据。

2 相关工作

本文的工作处于三条研究与工程脉络的交汇处：AI 编码代理、终端多路复用与自动化、以及 LLM 工具调用与自主代理循环。我们逐一梳理并指出 WhatyTerm 的差异化定位。

2.1 AI 编码代理

以 SWE-agent [2]、Devin 类端到端软件工程代理为代表的一批工作，聚焦于让 LLM 自主完成从问题理解到代码修改的闭环，并在 SWE-bench [1] 等基准上评估其解决真实 GitHub issue 的能力。这类系统通常自建“代理—环境”接口 (Agent-Computer Interface)，把文件编辑、命令执行等抽象为代理可调用的动作空间。与之并行，工业界的命令行编码代理 (Claude Code、Codex CLI、Gemini CLI 等) 则直接以人类使用的终端为运行环境。

WhatyTerm 与这两类工作的根本区别在于关注层次。SWE-agent 类工作关注“代理如何决策与修改代码”，而 WhatyTerm 不重新实现一个编码代理，而是关注“如何让已有的命令行编码代理在无人值守下持续运行”。换言之，WhatyTerm 是编码代理之上的编排与监督层：它把多个异构 CLI (Claude/Codex/Gemini 等) 视作黑盒执行体，通过感知其终端状态并注入操作来消除人工中断点。这一定位使其与具体编码代理解耦，可同时驱动多家 CLI。

2.2 终端多路复用与自动化

tmux、GNU screen 等终端多路复用器提供了会话持久化、断线重连与程序化控制（如 `send-keys`、`capture-pane`）能力，长期用于服务器运维与自动化脚本。传统的终端自动化工具（如 `expect`）则以“模式匹配—发送响应”的方式驱动交互式程序。

WhatyTerm 复用了 tmux 作为会话持久层——真正的代理进程活在 tmux 会话中，前端 PTY 仅作为可观察窗口，从而实现长任务的断连续跑与可回看。但其状态判断不再是 `expect` 式的静态模式匹配，而是**规则状态机与 LLM 语义分析相结合**的动态决策，且面对的是 LLM 编码代理特有的、由富文本 TUI 渲染的复杂屏幕。这使得 WhatyTerm 在“程序化终端控制”这一传统能力之上，叠加了针对 AI 代理交互模式的语义理解层。

2.3 LLM 工具调用与自主代理循环

ReAct [3] 提出的“推理—行动”交替范式，以及围绕工具调用 (function calling) 构建的各类自主代理框架，奠定了 LLM 驱动多步任务的基础。此类“感知—决策—行动”循环是自主代理的通用骨架，但朴素实现往往缺乏对长程任务全局结构的组织，容易陷入重复动作或无界循环。

WhatyTerm 在两个方面深化了这一范式。其一，在**决策层**引入分层结构，以确定性规则消化高频确定性状态，仅在不定时调用 LLM，兼顾成本与可控性——这与“每一步都调用 LLM”的朴素循环形成对比。其二，在**任务层**引入三层嵌套循环，通过需求分解、接口契约、拓扑排序并行、集成验证与失败根因分类，为长程任务施加显式的全局结构，并以每层独立的重试预算保证终止性。据我们所知，将“失败类型的语义分类”用于选择“最小修复范围（局部重跑 vs. 回退重拆）”，是本系统区别于通用代理循环的一个具体设计。

2.4 小结

综上，WhatyTerm 的差异化定位可概括为：它不是又一个编码代理，而是一个**面向异构命令行编码代理的、以分层决策与多层自主循环为核心的无人值守编排系统**，在传统终端多路复用能力之上叠加了 AI 语义感知，在通用代理循环之上叠加了成本可控的分层决策与结构化的长程任务组织。

3 系统架构

WhatyTerm 采用分层架构，自底向上依次为：执行/会话层、感知层、决策层、编排层与前端。图 1 给

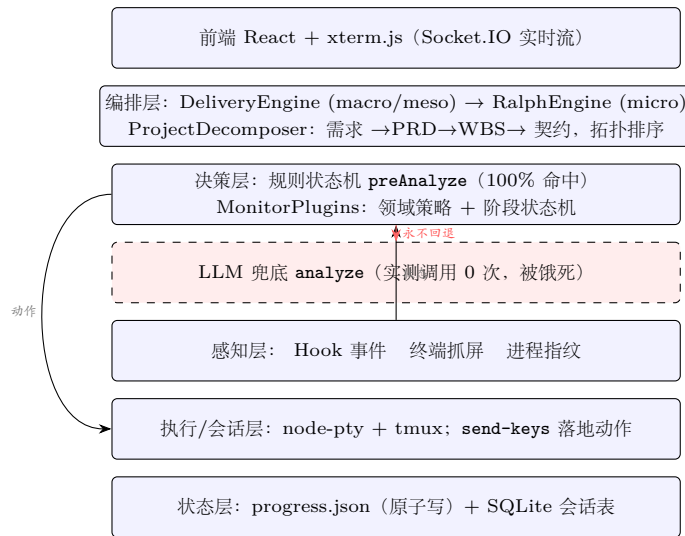


图 1: WhatyTerm 总体分层架构。红色虚线框为设计中的 LLM 兜底层，实测中从未被触发 (§6) ——控制流在规则层即闭环，永不下探。

出总体结构。本节先概述各层职责与数据流，再重点阐述作为系统基础的三通道感知设计。

3.1 会话层：tmux 作为持久化基底

系统以 Session 抽象统一管理终端会话，运行后端可为 macOS/Linux 上的 tmux、Windows 原生 PTY 或 mux-server 守护进程。以 tmux 为例，会话进程通过 `pty.spawn` 挂接到 `tmux attach-session`，真正的代理进程活在 tmux 中，前端 PTY 仅是“观察窗口”。这一设计带来两个关键性质：其一，前端断开或崩溃不影响会话，系统启动时以 `tmux has-session` 探测并恢复未关闭的会话；其二，所有对代理的操作都归结为 `tmux send-keys` 注入按键，从而将“LLM 的语义决策”与“终端的具体操作”彻底解耦。会话元数据持久化于 SQLite，运行态由 tmux 承载，不可恢复的旧会话另存于 `closed_sessions` 表供重启。

3.2 感知层：三通道信号融合

准确判断“代理此刻处于什么状态”是整个系统的前提。WhatyTerm 不依赖单一信号源，而是融合三路互补通道。

通道一：Hook 事件（精确时序）。主流编码代理提供官方 Hook 机制，可在工具调用前后 (`PreToolUse/PostToolUse`) 与会话停止 (`Stop`) 时主动推送事件。WhatyTerm 自动将 Hook 安装到 Claude Code、Gemini、Codex 等多家 CLI，并以 Claude Code 的事件模型为标准接口，通过适配器将其他 CLI 的事件名归一化（如 Gemini 的 `BeforeTool/AfterTool/SessionEnd`）。Hook 提供了

比屏幕解析更精确的运行/空闲边界：`PreToolUse` 到达即置会话为 `working`，`PostToolUse/Stop` 到达即置为 `idle`。该状态被自动化循环直接消费——处于 `working` 期间不打扰代理，避免误操作。

通道二：终端抓屏（语义内容）。Hook 只给出时序边界，具体“屏幕上在问什么”仍需解析文本。系统以 `tmux capture-pane -p -e` 周期性抓取屏幕内容，供决策层做语义分析。由于后台监控对每个会话每秒可能多次抓屏，若每次都同步 fork 子进程会周期性阻塞事件循环，系统实现了约 1.5 秒 TTL 的快照缓存与异步抓屏的并发去重（多个并发调用共享同一进行中的 `exec Promise`），将每秒的进程创建从数十次降至每会话约一次。

通道三：进程指纹（CLI 身份）。判断“当前跑的是哪家 CLI”优先采用进程级检测（检查 tmux 会话下的进程），失败时回退到终端文本指纹（为每种 CLI 预置一组特征正则）。准确的 CLI 身份使决策层能加载正确的状态判定规则。

三通道的融合使系统兼具精确的时序 (Hook)、丰富的语义 (抓屏) 与正确的身份 (进程)，显著降低了单一屏幕解析带来的延迟与误判。

3.3 会话精确归属

同一工作目录下可能同时运行多个编码代理实例，仅靠工作目录无法区分 Hook 事件应归属哪个会话。为此，Hook 脚本回传其所在的 `$TMUX_PANE`，系统优先通过 `tmux display-message` 反查该 pane 属于哪个 tmux 会话，从而实现事件到会话的精确归属；仅在没有 pane 信息时才回退到工作目录的最长前缀匹配。此外，Hook 脚本作为 CLI 的子进程，会继承 CLI 实际注入的环境变量，因而还顺带回传了当前生效的供应商 base URL 与模型等元数据，作为判断“当前使用哪个供应商”的最高优先级证据。

3.4 安全边界

由于 Hook 端点接收会触发自动操作的事件，系统为其生成 32 字节随机 Token（文件权限 0600），Hook 脚本仅从本机回环地址上报并携带该 Token，服务端逐一校验，防止外部伪造事件。会话名在用于 `tmux` 命令前经过清洗以防命令注入。这些机制构成了自动化操作的基本安全边界，更完整的权限与失败模式讨论见 §9。

4 核心方法

本节阐述三项核心方法：分层决策、可插拔领域策略，以及三层嵌套自主循环。

4.1 分层决策：规则优先、LLM 兜底

决策的目标是：给定当前终端屏幕文本与上下文，输出一个结构化决策，说明代理是否需要外部动作、需要何种动作。WhatyTerm 采用两级决策架构。

第一级：确定性规则状态机。一个纯规则函数按优先级从高到低对清洗后的屏幕文本做匹配，直接产出结构化决策，完全不调用 LLM。其判定优先级大致为：运行中 → 评分/菜单界面 → 权限确认界面 → 致命错误 → shell 空闲提示符 → 代理空闲 → 其他。只有当规则无法定论（返回空）时，才进入第二级。

第二级：LLM 兜底判官。对规则无法覆盖的开放场景，构造一份带优先级排序的判断规则提示词，强制模型只返回纯 JSON 决策。响应解析做了多重容错：剥离代码块标记、括号配对提取首个完整 JSON 对象、失败后重试修复，以应对模型输出不规范的情况。

无论哪一级，输出都遵循统一的动作模式（schema）：动作类型取自有限集合 { 文本输入, 菜单选择, 单字符, shell 命令, 确认, 警告, 自动修复, 无 }, 下游据此用 `tmux send-keys` 发送对应按键。系统内置计数器分别统计“规则预判”与“LLM 分析”的次数，使两级占比可被量化（见 §8）。

鲁棒状态识别。真实 TUI 屏幕是被 ANSI 转义序列、盒装 UI、滚动历史污染的脏文本，直接匹配极易误判。系统对 CSI/OSC/字符集切换等序列做多级清洗，并只对“最后若干行 / 最后若干字符”做窗口化匹配，避免历史缓冲区中的旧关键字干扰。一个典型的歧义消解案例是：Claude Code 空闲时底部仍常驻“`esc to interrupt`”状态栏，因此系统规定——只有在不存在空闲提示符时该状态栏才计为“运行中”，而真正可靠的运行标志是进度计时指示器（形如... (Ns)）。

4.2 可插拔领域策略框架

不同任务类型对“何时自动继续、何时必须停手让人确认、何为错误”的判断规则截然不同。硬编码单一规则无法覆盖，为此系统提供基于策略模式的可插拔插件框架。每个插件实现统一接口：`matches`（用项目特征正则判断是否适配当前项目）、`detectPhase`（识别当前处于项目的哪个阶段）、`getPhaseConfig`（返回该阶段的自动动作、检查点、告警模式、提示词模板与是否需人工确认）、`analyzeStatus`（据阶段给出决策）。插件管理器按序注册十余个内置领域策略（全栈开发、论文写作、数据分析、TDD、安全审计等），通用兜底策略最后注册；选择时遍历非兜底插件返回首个匹配者，皆不中则用兜底。

该框架的两个设计要点值得强调。其一，**提示词与代码分离**：每个插件每个阶段的提示词以独立 Markdown 文件存放，支持片段复用与变量替换，并可热重

表 1: 三层自主循环的职责与重试预算。

层	职责	重试预算
macro	需求 → PRD → WBS → 接口契约；根因回退重拆	2
meso	子任务完成后集成验证（装依赖/编译/测试）	3
micro	单个子任务：执行 → 验证 → 失败重试	5

载，构成一个模板化、可维护的领域知识库。其二，**阶段级自动化开关**：部分阶段（如“测试归档”）显式设置为禁止自动化、强制人工确认，将“人在环边界”形式化为阶段配置的一部分，而非散落在代码中的特判。

4.3 三层嵌套自主循环

对于“完成一个完整需求”这类长程目标，单代理反复试错缺乏全局结构。WhatyTerm 将自主交付组织为三层嵌套循环，如表 1 所示。

macro 层：需求到可执行任务包。需求分解经三步流水线，每步一次 LLM 调用：澄清（判断需求是否充分，不足则产出澄清问题）、生成结构化 PRD、生成 WBS。WBS 将需求拆成子任务并为每个子任务附上接口契约（输入、输出、验收标准、明确不包含的内容），要求“每任务规模受限、不跨层、依赖显式化、验收标准可机器验证”。随后按依赖关系对任务做拓扑排序，划分为可并行执行的批次，并处理循环依赖兜底。

micro 层：Developer/Validator 双角色。每个子任务在 micro 层由两个职责分离的角色处理：Developer 只负责实现、自检、提交并输出可复用经验；Validator 只负责逐条对照验收标准判定、不修改代码，并强制输出 `VALIDATION: PASS/FAIL` 及原因。这一“生成者/批判者分离”模式缓解了 LLM 自我裁判过宽的问题。子任务通过则标记完成，失败则计数重试，达上限则标记为 `blocked` 并跳过，防止卡死。算法 1 给出 micro 循环。

meso/macro 层：集成验证与根因回退。所有子任务按批次并行执行完成后，meso 层自动探测并运行安装/编译/测试命令做集成验证。若验证失败，系统调用 LLM 对失败做语义分类——归为接口不匹配、实现缺陷、设计缺陷或需求缺口之一——并据此选择最小修复范围：接口/实现类问题在 meso 层局部重跑受影响的子任务；设计/需求类问题则回退到 macro 层重新拆解。这种“以失败类型选择修复范围”的策略，是本系统对通用代理循环的一个具体深化。三层各自的重试预算共同保证了整个流程的终止性。

执行与完成检测。子任务以 `headless` 方式执行：将提示词写入临时文件，通过管道喂给 CLI 的非交互模式，并在命令末尾追加一个带返回码的完成标记；引擎轮询输出文件中的该标记与返回码来判定完成，而非

Algorithm 1: micro 层子任务循环

Input: 任务集合 T , 每任务最大重试 $R=5$

1 while 存在依赖已满足且未完成的任务

$t \leftarrow \text{NEXTTASK}(T)$ do

2 $out \leftarrow \text{RUNDEVELOPER}(t)$

3 $v \leftarrow \text{RUNVALIDATOR}(t, out)$

4 if $v = \text{PASS}$ then

5 | 标记 t 为 completed; 沉淀可复用经验

6 else

7 | $t.retry \leftarrow t.retry + 1$

8 | if $t.retry \geq R$ then

9 | | 标记 t 为 blocked (跳过)

10 | end

11 end

12 end

从屏幕检测——这避免了命令回显中出现的标记字样造成误判。同时设置首字节、静默与总时长三重超时兜底。任务改动在专属 git 分支上进行以隔离影响，且支持每任务暂停等待人工审查。

跨迭代经验沉淀。Developer 在完成任务时可输出“可复用经验”，经进度管理器累积后注入后续任务的上下文，形成一个轻量的经验记忆机制，使后续任务复用已有约定、减少重复踩坑。所有进度以临时文件加原子重命名的方式持久化，支持断点续跑。

5 软件工程应用

本节说明 WhatyTerm 在真实软件工程场景中的应用方式，并给出两个端到端的使用轨迹。

5.1 典型应用场景

无人值守的自动化操作。最基础的应用是让编码代理在无人看管下持续推进任务：自动化监督层感知到代理空闲或弹出确认框时，自动注入“继续”或选择“允许本次会话”，使一次可能需要数小时、涉及数十轮交互的开发任务无需人工全程值守。

全流程的构建—测试—交付。结合领域策略框架，系统可在“开发”阶段自动放行常规确认，而在“测试归档”等关键阶段强制停手等待人工确认；配合持续集成与发布流程，代理产出的改动经隔离分支提交后进入常规 CI，形成从编码到跨平台交付的链路。

自主项目交付。借助三层自主循环，用户只需给出一个较高层次的需求，系统即可自动完成需求澄清、任务拆解、并行开发、集成验证与失败回退，产出一个可验证的交付物。

5.2 案例一：交互式无人值守开发

考虑一个“为现有 API 端点增加分页能力”的任务。用户在某会话中启动编码代理并开启自动操作后离开。此后：代理读取相关文件、提出修改方案，在需要执行文件写入时弹出权限确认——监督层依据领域策略自动选择“允许本次会话”；代理编译并运行测试，若出现瞬时 API 限流错误，系统的供应商故障转移机制自动切换到下一供应商重试；代理完成后停在空闲提示符，监督层注入“继续”推进到下一子步骤；当代理输出“任务已完成”类措辞时，死循环防护逻辑识别到无新任务，停止注入并提示用户。整个过程中，人工干预点仅为任务开始时的启动与结束时的验收。

5.3 案例二：自主交付一个完整需求

考虑一个更高层次的需求。系统首先在 macro 层判断需求是否需要澄清，随后生成 PRD 与带接口契约的 WBS，并将子任务按依赖拓扑排序为若干可并行批次。每个子任务进入 micro 层，由 Developer 实现、Validator 逐条验收，失败则在预算内重试。全部子任务完成后，meso 层运行集成验证；若集成失败，系统对失败做语义分类并选择最小修复范围——例如判定为“接口不匹配”则仅局部重跑受影响的子任务，判定为“设计缺陷”则回退 macro 层重新拆解。整个过程的进度持续落盘，用户可随时在终端旁观代理的实时输出，也可在每个子任务后暂停审查。

5.4 与人工干预点的关系

上述两个案例共同体现了 WhatyTerm 的一个设计取向：自动化不追求消灭人工干预，而是将人工干预点显式化、最小化并可配置。哪些阶段可全自动、哪些阶段必须人工确认，由领域策略的阶段配置决定；这既保证了长程任务的自主推进，又在关键节点保留了必要的人类监督。

6 现实检验：一次真实部署的日志分析

前述章节描述了 WhatyTerm 的设计蓝图。本节转向一个更关键的问题：它真的这样工作吗？我们对系统在作者机器上一次真实部署的完整历史记录进行分析。数据来源为系统自身持久化的 SQLite 历史表 (history) 与自主执行引擎的输出文件，均为运行时自动记录、非事后构造。

表 2: 四个会话的自动决策计数（共 46 次）。

会话（目标摘要）	决策数
afad3bb2（自主开发迭代）	36
83993863（论文写作）	4
630f0238（arXiv 论文构建）	3
8bf5eba6（论文写作）	3
合计	46

表 3: 46 次决策的动作构成。

动作	次数	占比
“继续”	40	87%
重启 CLI (<code>claude -c/claude</code>)	5	11%
真实任务指令	1	2%

6.1 数据集

该部署跨越 2026 年 7 月 17 日至 20 日，共三天。历史表含 62 条记录，其中系统事件 9 条、终端输出 7 条、自动决策 (`ai_decision`) 46 条。这些决策分布于四个会话，其目标均与“将本 harness 构建为一篇论文/软件工程任务”相关。表 2 给出各会话的决策计数。

6.2 发现一：规则层 100% 垄断决策

最直接的发现是：全部 46 次自动决策，无一例外地由规则层（标记为“预判断自动操作”）做出；LLM 兜底层的调用次数为零。按 §4.1 的定义，规则命中率

$$\rho = \frac{N_{\text{pre}}}{N_{\text{pre}} + N_{\text{llm}}} = \frac{46}{46 + 0} = 100\%.$$

在原始设计叙事里，高 ρ 被视作“以零 Token 处理多数常见状态”的成本优势。但一个 100% 的 ρ 意味着截然不同的事情：系统从未认为任何一次情形复杂到需要动用 LLM 的语义判断。设计中作为“智能兜底”的第二级决策，在整个部署周期内是完全的死代码路径。

6.3 发现二：87% 的动作只是“继续”

进一步看 46 次决策的动作构成（表 3）：“继续”占 40 次（87%），重启 CLI (`claude -c` 或 `claude`) 占 5 次，而携带真实语义的任务指令——形如“请继续完成任务 7/15 个任务：实现……”——仅出现 1 次（2%）。

这组数字刻画了一个令人不安的运行画像：在近九成的时间里，这个“自主开发编排系统”所做的，不过是反复向终端敲入“继续”二字。它既没有理解代理当前卡在何处，也没有判断这一声“继续”是否真的推动了任务——它只是在“看到空闲提示符”这一条件反射下，机械地重复同一个动作。

6.4 发现三：自主引擎从未真正交付

与交互式决策的空转相呼应，同期的三层自主引擎（RalphEngine）表现同样失败。我们检查其 headless 执行的输出文件：早期批次（十余个）输出为空，仅有一行完成标记；后期批次则直接写入 `Execution error`——这是被调用的 CLI 启动即失败时打印的错误，而非引擎的正常产出。整个部署期间，没有任何一个自主子任务真正产出过内容。引擎因此不断重试、生成新的执行命令，形成一个“生成命令—执行失败—再生成”的空循环。

6.5 小结：一个退化为宏的自主系统

三个发现指向同一结论：无论是交互式监督路径（退化为反复“继续”），还是自主执行路径（反复失败空转），WhatyTerm 在这次真实部署中都未能体现其设计所承诺的“智能编排”。它在功能上退化为一个昂贵的“自动点继续”宏。值得强调的是，这并非因为系统崩溃或报错——恰恰相反，它“运行良好”地、稳定地做着无用功。这种“看似正常运行、实则空转”的形态，比显式崩溃更危险，也更值得剖析。下一节我们深入代码，解释这一退化为何会稳定发生。

7 根因分析：规则层为何吞噬了智能

§6 的现象需要一个机制层面的解释：为什么 LLM 兜底层永不触发？为什么系统满足于反复“继续”？本节回到代码，给出确切答案，并将其抽象为一个可推广的反模式。

7.1 “检测到提示符即继续”的无条件路径

决策的核心入口先调用规则层 `preAnalyzeStatus`，仅当其返回空时才回退到 LLM。问题的关键在于：规则层对“空闲”这一最高频状态，给出了一条无条件的、必定返回非空的判定路径。

具体而言，通用领域插件的状态机在识别到终端存在空闲提示符（形如行首的 `>`）且不处于运行中时，将阶段判定为 `waiting`，并直接返回一个固定动作：

```
// phase === 'waiting'
{ needsAction: true,
  actionType: 'text_input',
  suggestedAction: '继续',
  message: '检测到等待输入状态,发送"继续"指令' }
```

这条路径的判定条件仅是“存在空闲提示符”这一纯语法信号，与“代理上一步做了什么、这一步该做什么、继续是否有意义”完全无关。由于长程任务中“代理停在空闲提示符”是压倒性的高频状态，这条路径几乎在每一个决策时刻都被命中并返回非空结果——于是控制流

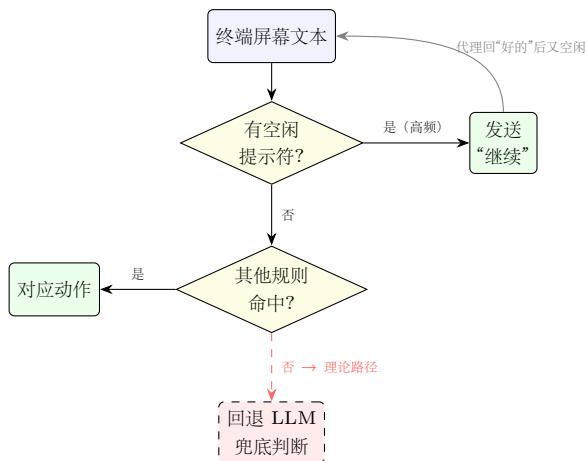


图 2: 决策控制流。空闲提示符是长程任务中的压倒性高频状态,“是”分支几乎每次命中并发送“继续”;代理敷衍回应后重新空闲,形成灰色反馈回路。红色虚线的 LLM 回退路径在实测中从未被走到。

永远不会走到那个 `return null`, LLM 兜底层因此永远得不到调用的机会。

图 2 刻画了这一控制流。系统确实内置了少量“更聪明”的空闲子判定:例如识别代理明确输出“待命中/需要具体任务”时停手(避免“继续 → 待命 → 继续”死循环),或识别“列出多个下一步方向”时回复“按建议顺序继续”。但这些都是更窄的正则特判,命中后同样在规则层内闭环返回,并不构成向 LLM 的移交。它们扩充了规则层的“聪明程度”,却没有改变“规则层垄断决策、LLM 永不介入”的结构。

7.2 缺失的一环：动作有效性反馈

更深层的缺陷是:规则层发出“继续”后,没有任何机制评估这一动作是否真正推进了任务。系统只观察“屏幕是否回到空闲提示符”,而这在“继续”无效时同样成立——代理回一句“好的,我继续”然后又停下,屏幕状态与真正干活后停下无法区分。缺少“任务进度”这一反馈信号,系统就无法察觉自己陷入了空转,也就没有任何触发条件把决策权上交给能理解语义的 LLM。

形式化地说,理想的分层决策应满足一个移交条件:当规则层的动作在连续 k 次后未带来可观测的任务进展时,应强制回退到 LLM 层重新判断。而现有实现的移交条件恒为假——规则层对高频状态总有一个自信的、语法驱动的答案,从不承认“我不确定”。

7.3 一个可推广的反模式

我们将这一现象抽象为自主代理 harness 设计中的“廉价层垄断”反模式:

当一个分层决策系统把高频状态的最终决定权交给廉价的规则层,且规则层的判定基于纯

表 4: 效率指标 (真实值) 与其误导性解读。

指标	实测值	表面解读 vs. 真相
规则命中率 ρ	100%	“省成本”/ 实为智能缺位
LLM 调用次数	0	“高效”/ 推理层被饿死
决策延迟	毫秒级	“响应快”/ 只是正则匹配
崩溃次数	0	“稳定”/ 稳定地空转

语法信号、缺乏动作有效性的反馈时,昂贵但智能的推理层将被系统性地饿死;系统会稳定收敛到“高命中率、低有效性”的空转,且因不报错而难以被察觉。

这一反模式的隐蔽性在于其指标伪装:规则命中率 ρ 、决策延迟、Token 成本等常规指标全都“表现优异”(ρ 高、延迟低、成本近零),恰恰因为系统根本没在做有意义的工作。任何仅依赖这些效率指标的评估都会给出“系统运行良好”的错误结论——这也是为什么 §8 必须引入任务推进度这类有效性指标,而非仅看效率指标。

7.4 修复方向

根因清晰后,修复方向也随之明确(详见 §9):其一,为“继续”类动作引入有效性反馈,例如比对动作前后屏幕内容的实质变化、或利用 Hook 事件确认代理确实发起了新的工具调用;其二,设定规则层向 LLM 层的强制移交条件,如“连续 k 次继续未见新工具调用”即上交;其三,区分“语法空闲”与“语义空闲”——前者仅说明有提示符,后者才说明代理确实无事可做。这些改动的共同目标,是打破规则层对决策权的垄断,让推理层在真正需要时得以介入。

8 评估：效率指标的误导性

§6 已给出核心的实测结果。本节从评估方法论的角度做一层反思:为什么常规的效率指标不仅无法发现前述退化,反而会掩盖它?并给出应当采用的有效性指标。

8.1 效率指标全部“通过”

若以自主代理系统常用的效率指标衡量这次部署,结论会是全面正面的(表 4):规则命中率 $\rho = 100\%$,意味着几乎零 LLM 成本;决策延迟极低(规则匹配为毫秒级);无崩溃、无未捕获异常,系统连续运行三天。任何只看这一组指标的评估都会判定“系统高效、稳定、经济”。

这正是 §7 所述“指标伪装”的具体体现:效率指标度量的是“系统做一件事有多快多省”,而完全不关心“这

件事是否有意义”。

8.2 应采用有效性指标

要暴露退化，必须引入度量任务是否真正推进的指标。我们定义并在本次数据上测量如下三项：

有效动作率。定义为“带来可观测任务进展的动作数 / 总动作数”。以“注入了真实语义内容”为宽松代理指标，本次部署中携带真实任务指令的动作仅 1 次，有效动作率约 $1/46 \approx 2\%$ 。即便把 5 次 CLI 重启也算作“有意图的动作”，上限也不过 $6/46 \approx 13\%$ 。

自主交付成功率。定义为“真正产出内容的自主子任务数 / 尝试数”。据 §6 发现三，RalphEngine 的 headless 执行无一产出内容，该指标为 0%。

空转比例。定义为“未带来进展的重复动作占比”。40 次“继续”中绝大多数属于此类，空转比例接近 87%。

三项有效性指标（约 2% 有效动作、0% 自主交付、87% 空转）与三项效率指标（ $\rho = 100\%$ 、0 次 LLM、0 崩溃）构成尖锐对比。同一次部署，在两套指标下得出完全相反的结论。这一对比本身就是本文的一个方法论贡献：评估自主代理系统时，效率指标必须与有效性指标并列，否则“高效的空转”会被误报为成功。

8.3 有效性威胁

样本规模：本文基于单次、单机、46 次决策的部署，规模有限，不足以支撑统计性结论；其价值在于提供一个有代码根因支撑的、可复现的负面案例，而非普适的量化率。**代理指标近似：**“有效动作率”以“是否注入真实语义内容”近似“是否推进任务”，可能低估或高估真实有效性；更严格的度量需要对任务状态做独立标注。**外部有效性：**退化的具体触发依赖于所用 CLI 的界面与规则实现，其他配置下的表现可能不同，但 §7 所抽象的“廉价层垄断”反模式不依赖具体实现。

9 讨论、局限性与可复现性

9.1 经验教训

这次“设计良好却在实测中退化”的经历，给出几条超出本系统的教训。

教训一：分层决策必须内建向上移交的强制条件。分层的初衷是“廉价层处理简单情形、昂贵层处理复杂情形”，但若廉价层对高频情形总能给出一个自信的答案，分层就退化为“廉价层独裁”。有效的分层必须让廉价层能够并且被迫承认不确定——例如以“连续 k 次动作无进展”作为强制回退到推理层的条件。缺了这一环，越廉价、越自信的规则层，越会把智能层饿死。

教训二：动作必须有有效性反馈，而非仅有状态触发。现有系统是纯粹的“状态 $\rightarrow$ 动作”反射：见到空闲提示符就发“继续”。它缺少“动作 $\rightarrow$ 效果”的闭环——没有观察“继续”是否真的带来了进展。任何自动化系统一旦缺失效果反馈，就无法区分“有用的重复”与“空转的重复”，从而稳定地滑向后者。

教训三：评估自主系统要以有效性指标为主、效率指标为辅。如 §8 所示，效率指标（命中率、延迟、成本、崩溃率）会在系统空转时集体“亮绿灯”。只有度量“任务是否真正推进”的有效性指标，才能戳破“高效空转”的假象。

教训四：不报错的失败比报错的失败更危险。系统三天里没有崩溃、没有异常，却也没做任何有用的事。这种“沉默的失败”难以通过监控告警发现，需要专门的进展度量与人工抽检才能暴露。

9.2 设计权衡与修复

§7 已指出修复方向：为“继续”类动作引入基于屏幕实质变化或 Hook 工具调用的有效性反馈；设定规则层向 LLM 层的强制移交条件；区分“语法空闲”与“语义空闲”。这些修复的共同代价是引入更多状态跟踪与可能的 LLM 调用，从而抬高成本——但这正是恢复“智能”所必须支付的价格。当前实现选择了成本最优（近零 LLM），代价是有效性归零；合理的设计应在二者间取得平衡，而非把 ρ 最大化本身当作目标。

9.3 安全边界与失败模式

自动化操作终端本质具有风险：系统代替人点击“确认”“继续”，误判即误操作。系统设置了 Token 鉴权、本机回环、会话名清洗、关键阶段强制人工确认、自主执行隔离分支等边界。本次部署未观察到误操作恰恰是因为它几乎没做实质操作——这提示我们，一旦有效性提升、系统真正开始自主行动，安全边界将面临远比现在更严峻的考验。

9.4 局限性

其一，本文的实证基于单次、单机、46 次决策的部署，样本有限，结论定位为“有根因支撑的负面案例”而非普适量化率。其二，规则状态机与所用 CLI 的界面耦合，CLI 升级可能改变退化的具体形态。其三，系统的自主执行引擎核心以闭源形式分发，开源包不含该核心而降级运行，这对完全基于开源代码的复现构成约束。

9.5 可复现性说明

前端、会话管理、感知层、决策层（含本文剖析的规则状态机）与领域策略框架均在开源仓库中可查证。本文的关键证据——46 次决策的历史记录与引擎输出文件——由系统运行时自动持久化于 SQLite 与临时文件，分析脚本仅做只读统计，可在相同数据上复现。三层自主循环的编排逻辑开源可见，但其执行引擎核心闭源，故涉及“自主交付是否成功”的复现需依赖正式发行版。为保证可复现，应记录仓库地址、版本 tag、CLI 与模型版本及运行平台。

10 结论

本文介绍了 WhatyTerm——一个面向命令行编码代理的无人值守终端编排系统——并以一次真实部署的实测数据为核心，报告了其设计意图与实际行为之间的落差。系统在设计上具备规则优先、LLM 兜底的分层决策，三通道感知，以及三层嵌套自主循环等机制；但跨三天、四会话、46 次决策的日志分析表明：全部决策 100% 由规则层做出、LLM 兜底从未触发，87% 的动作只是机械“继续”，自主引擎无一真正交付。一个本应智能编排自主开发的系统，退化成了“自动点继续”的宏。

我们将根因定位到规则层“检测到空闲提示符即无条件继续”的判定路径，并将其抽象为“廉价层垄断”反模式：当廉价规则层对高频状态总能给出自信的、语法驱动的答案，且缺乏动作有效性反馈时，昂贵的推理层会被系统性饿死，系统稳定收敛到“高效率、零有效性”的沉默空转。我们进一步指出，常规效率指标会掩盖这一退化，唯有引入任务进度等有效性指标才能将其暴露。

本文的立场是：一个诚实的负面结果，连同其可复现的证据与代码级根因，比一份夸耀设计优越的报告更有价值。我们希望“廉价层垄断”这一反模式、以及“效率指标须与有效性指标并列”的评估主张，能为自主代理 harness 的设计者提供一面镜子：真正的挑战不在于让系统跑起来，而在于让它在跑的时候确实在做有意义的事。未来工作包括实现并验证本文提出的移交条件与有效性反馈机制，并在更大规模的部署上检验退化是否被消除。

参考文献

- [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International*

Conference on Learning Representations (ICLR), 2024.

- [2] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.